



SERVERLESS AIR QUALITY INDEX (AQI) FORECASTING PIPELINE FOR LAHORE

An End-to-End Machine Learning System for Real-Time Air Quality Forecasting

Project Report

Prepared By	Muhammad Waleed
Email	waleedusman766@gmail.com
Contact	0334-4371256
Institution	Bahria University Lahore — BSIT
Repository	github.com/waleedusman747/aqi-forecast
Live Dashboard	aqi-forecast-tbkhapelgeb2kgiere7kpn.streamlit.app

Table of Contents

1. Executive Summary	4
2. Introduction and Objectives.....	4
3. System Architecture	5
4. Technology Stack.....	6
5. Data Source Selection and API Evaluation	6
5.1 AQICN — Investigated and Rejected.....	6
5.2 Open-Meteo — Used, Then Superseded	7
5.3 IQAir Community API — Final Live Data Source.....	7
6. Data Collection, Feature Engineering, and the Feature Store	8
6.1 Feature Pipeline	8
6.2 Engineered Features	8
6.3 Historical Backfill	8
6.4 Feature Store — Feast.....	8
7. Model Development and Training	9
7.1 Effect of Training Data Volume	9
8. Final Model Results and Comparison	10
8.1 Reasoning Behind Final Model Selection	10
9. Model Explainability (SHAP).....	11
9.1 Feature Importance — 24-Hour Forecast	11
9.2 Feature Importance — Longer Horizons	12
9.3 Feature Impact Distribution	12
10. Exploratory Data Analysis.....	13
10.1 Key Finding	13
10.2 Other Observations	14
11. Automation and CI/CD Pipeline	14
12. Web Application Dashboard	15
13. Blockers Encountered and How They Were Resolved	16
13.1 Hopsworks Feature Store — Windows Client Incompatibility.....	16
13.2 AQICN's Pakistan Monitoring Network Was Non-Functional	17
13.3 OneDrive Locking Git Files.....	17
13.4 Recurring Git "Deletion Failed" Loop	17
13.5 GitHub Actions Push Permission Denied.....	17
13.6 Broken Virtual Environment After Folder Move	18
13.7 Timezone Bug Causing an Inflated "Current" AQI.....	18
13.8 Git Rebase / Vim Editor Freezing the Terminal	18

13.9 Streamlit Dashboard Card Rendering Bug.....	18
14. Limitations and Future Work	19
15. Conclusion	20
16. References and Data Sources.....	20
17. Appendix — Project Links.....	20

1. Executive Summary

This report documents the design, development, and deployment of a fully serverless, end-to-end machine learning pipeline that forecasts the Air Quality Index (AQI) for Lahore, Pakistan, up to 72 hours in advance. The system was built to satisfy the requirements of a modern MLOps-style project: automated data collection, feature engineering, a feature store, model training and evaluation, explainability, exploratory analysis, and a live, publicly accessible dashboard — all without provisioning or maintaining a dedicated server.

The final system collects hourly air quality and weather data automatically via GitHub Actions, stores engineered features in a Feast feature store, trains Ridge Regression and Random Forest models to forecast AQI at 24-, 48-, and 72-hour horizons, explains its predictions using SHAP, and serves live forecasts, trend charts, and hazardous-air-quality alerts through a Streamlit web application deployed on Streamlit Cloud.

Development was not a straight line. Several planned components in the original specification — most notably the Hopsworks feature store and the AQICN data API — turned out to be non-viable in practice, for reasons outside the project's control (a Windows-specific client incompatibility in the former, and a dead monitoring-station network in the latter). Each blocker is documented in detail in Section 13, along with the investigation that led to the fix and the reasoning behind each replacement technology. The project's key quantitative finding — that 57.4% of hourly AQI readings over the past year in Lahore exceeded the "Unhealthy" threshold — underscores the practical value of the alerting system built into the dashboard.

2. Introduction and Objectives

Air pollution in Lahore regularly reaches hazardous levels, particularly during the winter smog season. Residents and public-health planners benefit from being able to see not just the current air quality, but a short-term forecast of how it will change over the coming days, so that outdoor activity, school closures, and health precautions can be planned in advance.

The objective of this project was to design and build a complete, self-updating machine learning system — a "serverless" pipeline — that:

- Automatically collects hourly air quality and weather data for Lahore, with no manual intervention.
- Engineers meaningful predictive features (time-based, lag-based, and rolling statistical features).
- Stores those features in a proper feature store for reproducibility and reuse.
- Trains and evaluates multiple regression models to forecast AQI 24, 48, and 72 hours ahead.

- Explains model predictions using SHAP, so that the reasoning behind a forecast is transparent.
- Performs exploratory data analysis to surface real patterns in Lahore's air quality.
- Presents the results through a live, interactive, publicly accessible web dashboard with built-in alerts for hazardous conditions.
- Automates the entire pipeline end-to-end using CI/CD (GitHub Actions), requiring no dedicated server or manual runs.

The following sections describe the architecture, the technologies evaluated and chosen, the data pipeline, the modelling approach and results, the explainability and analysis components, the challenges encountered during development, and the final deployed product.

3. System Architecture

The system follows a linear, event-driven pipeline. An hourly trigger (GitHub Actions) fetches new data, engineers features, and updates the feature store; separately, the training pipeline (run as needed) reads from the feature store to produce updated models; and the dashboard reads the latest features and the trained models to serve live forecasts. Figure 1 summarises this flow.

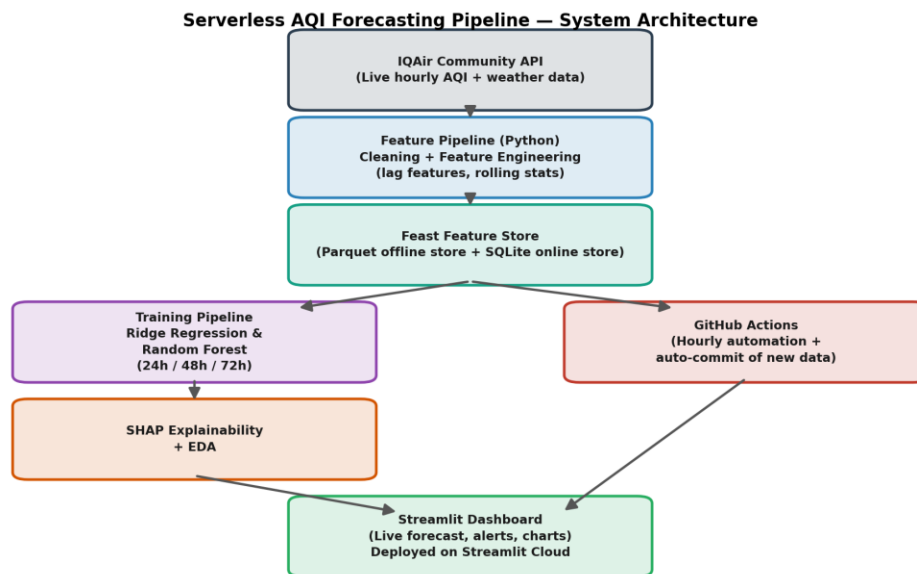


Figure 1 — High-level system architecture of the serverless AQI forecasting pipeline.

This design keeps every component decoupled: the data pipeline does not need the dashboard to be running, the dashboard does not need to retrain models to serve a forecast, and the entire system runs on free-tier cloud infrastructure (GitHub Actions for compute, Streamlit Cloud for hosting, Feast with local Parquet/SQLite for storage) with no persistent server to provision, patch, or pay for — satisfying the "serverless" requirement of the project.

4. Technology Stack

The table below lists every major component of the final system, and the specific technology used for it.

Component	Technology Used	Notes
Programming Language	Python 3.11	Chosen over the default Python 3.14 due to package compatibility issues (see Section 13).
Live Data Source	IQAir Community API	Final choice after AQICN and Open-Meteo were evaluated and found lacking (see Section 5).
Historical Backfill	Open-Meteo Air Quality & Weather Archive API	Free, no API key, supports 1-year historical range.
Feature Store	Feast (open-source)	Replaced Hopsworks after a Windows-specific client failure (see Section 13).
Offline Store	Parquet files	Versioned in the Git repository.
Online Store	SQLite	Feast's default local online store.
ML Framework	scikit-learn	Ridge Regression and Random Forest Regressor.
Explainability	SHAP (TreeExplainer)	Feature importance and summary plots per forecast horizon.
Dashboard	Streamlit	Interactive web app with dark/light themes.
Automation / CI-CD	GitHub Actions	Hourly cron-triggered workflow; also supports manual runs.
Version Control	Git & GitHub	Full commit history of code, models, and data.
Deployment	Streamlit Community Cloud	Public, free hosting directly from the GitHub repository.

5. Data Source Selection and API Evaluation

One of the most consequential technical decisions in this project was the choice of live data source. The original plan, matching the assignment specification, was to use the AQICN API. In practice, this required a systematic investigation across three candidate APIs before a reliable source was found.

5.1 AQICN — Investigated and Rejected

AQICN was tested first, since it is explicitly listed as an acceptable data source. The API call itself worked and returned a well-formed response, but closer inspection of the response timestamp

revealed that Lahore's only monitoring station ("Lahore US Embassy") had not reported new data since February 2025 — the API was silently returning a stale, cached reading rather than an error.

To confirm this was not an isolated fault, the investigation was extended:

- Karachi (US Consulate station) — last reported March 2025 (dead).
- Islamabad (US Embassy station) — last reported February 2026, roughly six months stale.
- A geo-coordinate nearest-station search around Lahore — returned a live station, but located in Delhi, India, over 500 km away.
- A search of AQICN's secondary "Punjab Network" station IDs — returned a live station, but located in Amritsar, India.

This confirmed that AQICN's entire Pakistan monitoring network (built on U.S. State Department embassy/consulate air-quality monitors) was non-functional at the time of development, across every major city tested — not a Lahore-specific or temporary issue. This is documented as a real-world data-infrastructure limitation discovered during the project, rather than a project defect.

5.2 Open-Meteo — Used, Then Superseded

Open-Meteo's satellite/model-based air-quality reanalysis (CAM5) was adopted as the replacement: it requires no API key, has no request-rate restrictions, and — critically — supports historical queries beyond its documented 92-day limit by passing explicit `start_date` and `end_date` parameters, which was used to backfill a full year of hourly training data (see Section 6.3).

However, once the dashboard went live, its "current AQI" reading (201) was cross-checked against multiple independent sources for the same day: the Punjab EPA official monitor (160), AccuWeather (124), IQAir (approx. 105–137), and Plume Labs (105). Open-Meteo's satellite-derived estimate was consistently 40–90 points higher than ground-based readings — a known characteristic of reanalysis models in cities with highly localised pollution sources, where a single regional estimate cannot capture block-by-block variation.

5.3 IQAir Community API — Final Live Data Source

IQAir's free "Community" API tier (api.airvisual.com, 10,000 calls/month, far more than the 24/day the hourly pipeline requires) was tested and found to closely match the other independent, ground-truth-anchored sources (approx. 106–124 for Lahore on the test date). IQAir achieves this accuracy the way most commercial air-quality platforms do: by fusing government stations, citizen sensors, and satellite estimates internally, rather than relying on any single feed — an approach this project could not fully replicate given its scope, but could approximate by selecting IQAir as the most reliable single upstream source available for free.

The live feature pipeline (`feature_pipeline.py`) was therefore switched to IQAir as its primary data source, while Open-Meteo remains in use for one-time historical backfilling, since a full year of IQAir history is not available on the free tier.

6. Data Collection, Feature Engineering, and the Feature Store

6.1 Feature Pipeline

The feature pipeline (`feature_pipeline.py`) runs every hour. On each run it: (1) calls the IQAir API for the latest AQI and weather reading for Lahore; (2) computes derived features; (3) appends the new row to a Parquet dataset, replacing any row with a duplicate timestamp; and (4) materialises the new data into Feast's online store so it is immediately queryable.

6.2 Engineered Features

Two families of features were engineered:

Time-based features: hour of day, day of week, month, and a weekend flag — capturing daily and seasonal traffic/industrial patterns.

Lag and rolling-window features: AQI 1, 6, and 24 hours prior; 6- and 24-hour rolling means; a 6-hour rolling standard deviation; and a 1-hour AQI change rate. These were added after an initial round of training showed that weather variables alone were insufficient — AQI is highly autocorrelated, and its own recent history is the single strongest predictor (confirmed later by the SHAP analysis in Section 9).

6.3 Historical Backfill

To train a usable model, a substantial historical dataset was required. Rather than waiting weeks for the hourly pipeline to accumulate data, a one-time backfill script (`backfill.py`) queried Open-Meteo's archive endpoints for a full year of hourly air-quality and weather data (approximately 8,760 hourly rows, reduced to 8,788 valid rows after cleaning), giving the training pipeline sufficient history to learn seasonal and daily patterns.

6.4 Feature Store — Feast

Feast was used as the feature store, with a local Parquet file as the offline store and a SQLite database as the online store. An entity (`city`) and a feature view (`aqi_features_view`) were defined covering all pollutant, weather, and time-based fields. This satisfies the feature-store requirement of the assignment while avoiding the Windows-incompatibility that ruled out Hopsworks (see Section 13.1).

7. Model Development and Training

The training pipeline (`training_pipeline.py`) builds a separate supervised-learning problem for each forecast horizon (24, 48, and 72 hours ahead), by shifting the AQI column forward by the corresponding number of hours to create the prediction target. Two models were trained and compared for every horizon:

- Ridge Regression — a simple, regularised linear baseline.
- Random Forest Regressor — an ensemble tree-based model (200 estimators, max depth 10).

Data was split chronologically (80% train / 20% test) rather than randomly shuffled, since random shuffling on time-series data leaks future information into the training set and produces misleadingly optimistic results. Each model was evaluated using three standard regression metrics: RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), and R^2 (coefficient of determination).

7.1 Effect of Training Data Volume

Training was performed twice: first on an initial ~60-day dataset (1,560 rows), and again after the full one-year backfill (8,788 rows). The additional data produced a substantial improvement across every horizon, as shown in Figure 2.

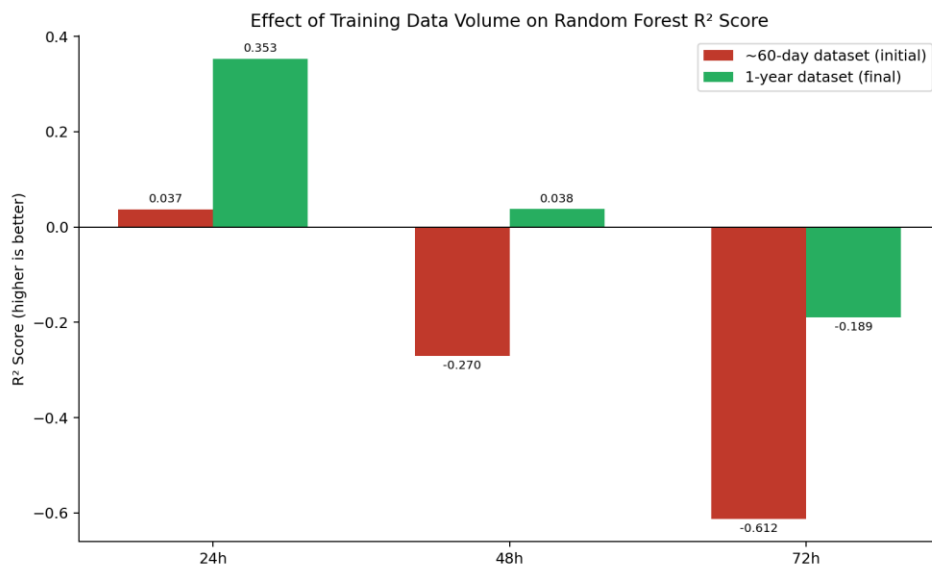


Figure 2 — R^2 improvement after increasing the training dataset from ~60 days to one full year.

With only 60 days of data, both the 48-hour and 72-hour models had negative R^2 scores — meaning they performed worse than simply predicting the historical average. After training on a full year of data, the 24-hour model became clearly usable ($R^2 = 0.353$), the 48-hour model crossed into positive territory ($R^2 = 0.038$), and the 72-hour model improved substantially (from $R^2 = -0.612$ to $R^2 = -0.189$) though it remains a known limitation (see Section 14).

8. Final Model Results and Comparison

The table below shows the final evaluation metrics for both models, at all three forecast horizons, trained on the full one-year dataset. The best-performing model for each horizon (lowest RMSE) is marked and was the one saved and deployed to the dashboard.

Horizon	Model	RMSE	MAE	R ²	Selected
24h	Ridge Regression	30.73	21.47	0.198	
	Random Forest	27.61	19.22	0.353	✓
48h	Ridge Regression	39.23	28.27	-0.303	
	Random Forest	33.70	24.22	0.038	✓
72h	Ridge Regression	41.87	31.28	-0.481	
	Random Forest	37.51	26.82	-0.189	✓

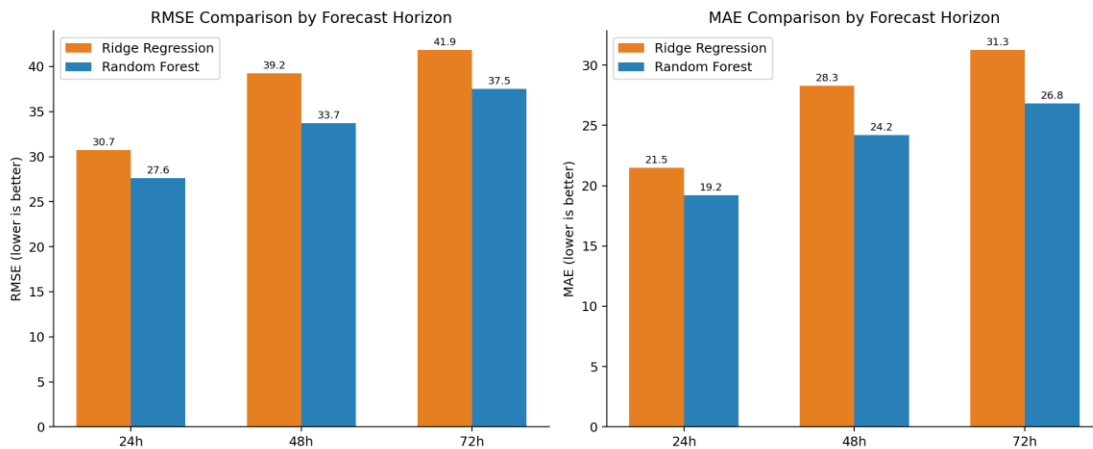


Figure 3 — RMSE and MAE comparison between Ridge Regression and Random Forest across all forecast horizons.

8.1 Reasoning Behind Final Model Selection

Random Forest was selected as the production model for all three horizons, for the following reasons:

- It outperformed Ridge Regression on every single metric, at every horizon, with no exceptions.
- AQI's relationship with its own lag features and weather variables is non-linear (e.g. pollutant build-up under specific pressure/humidity conditions is not a straight-line effect), which a tree-based ensemble captures naturally, while a linear model like Ridge cannot.

- Random Forest's built-in feature importance and native SHAP TreeExplainer support directly enabled the explainability component required by the project (Section 9).
- The gap between Ridge and Random Forest widened as the horizon increased, suggesting Ridge's linear assumption breaks down further into the future — reinforcing Random Forest as the more robust choice for longer-range forecasting.

The 24-hour Random Forest model is considered production-ready for practical use. The 48-hour model is usable but should be treated with more caution. The 72-hour model, while it is the best available option among those tested, still has a negative R^2 and is presented in the dashboard with that limitation transparently acknowledged (see Section 14).

9. Model Explainability (SHAP)

SHAP (SHapley Additive exPlanations) was used to explain what each Random Forest model was actually basing its predictions on. A TreeExplainer was applied to the 200 most recent rows of feature data for each horizon, producing both a feature-importance bar chart and a feature-impact summary plot per horizon.

9.1 Feature Importance — 24-Hour Forecast

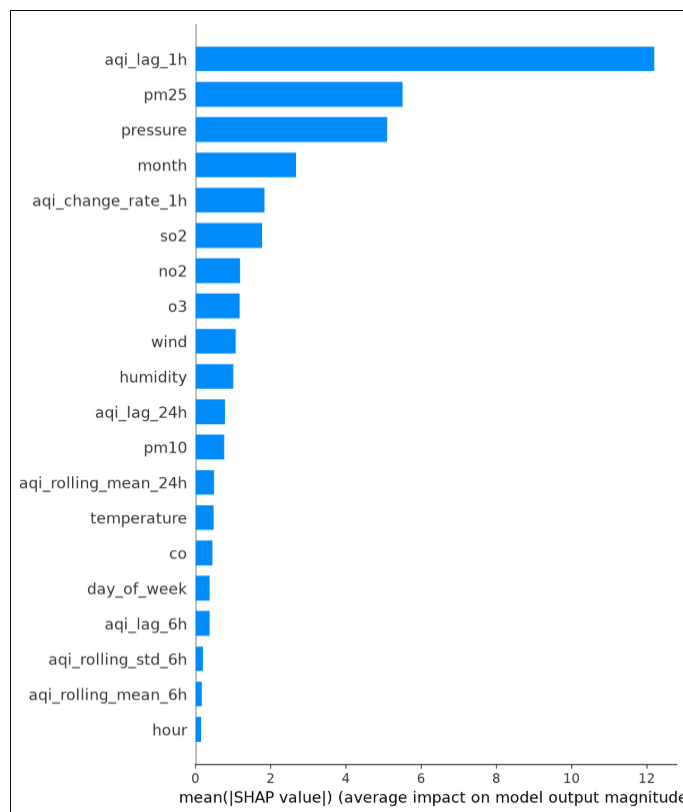


Figure 4 — SHAP feature importance for the 24-hour forecast model.

For the 24-hour horizon, the single most influential feature by a wide margin is `aqi_lag_1h` — the AQI reading one hour prior — followed by PM2.5 concentration and atmospheric pressure. This is intuitive: air quality does not change drastically hour-to-hour, so the most recent reading is the strongest anchor for a near-term forecast.

9.2 Feature Importance — Longer Horizons

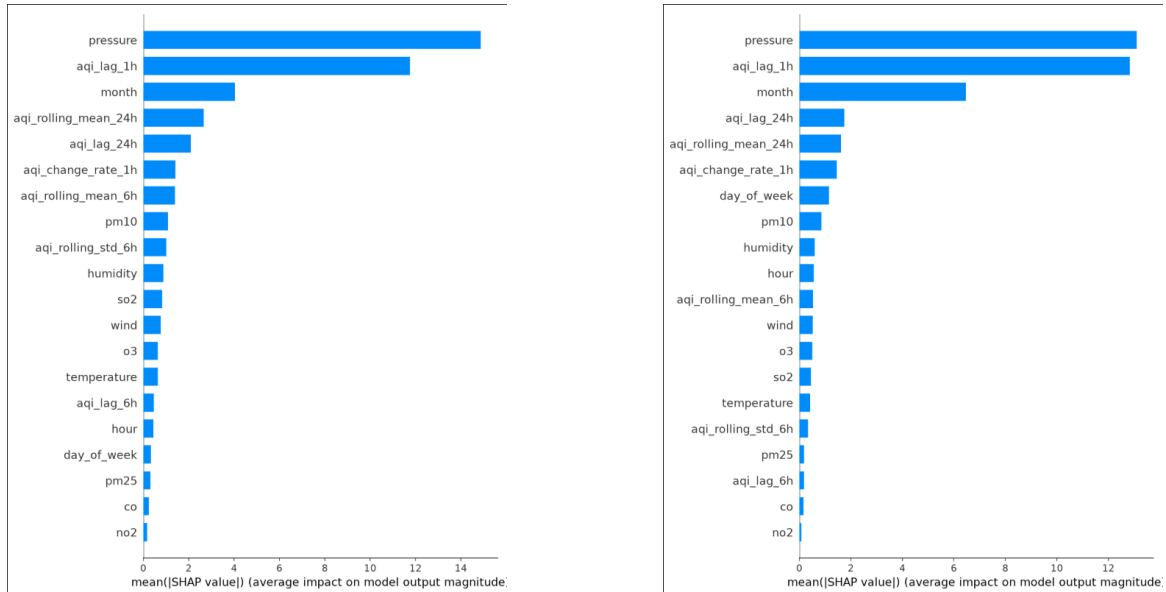


Figure 5 — SHAP feature importance for the 48-hour (left) and 72-hour (right) forecast models.

A clear and genuinely interesting shift occurs at the longer horizons: atmospheric pressure overtakes `aqi_lag_1h` as the most important feature for both the 48-hour and 72-hour models. This is a meaningful finding rather than a coincidence — pressure systems are associated with smog-trapping temperature inversions and are more predictive of how conditions will evolve over multiple days than a single recent AQI reading, whose influence naturally decays the further into the future the model is asked to predict.

9.3 Feature Impact Distribution

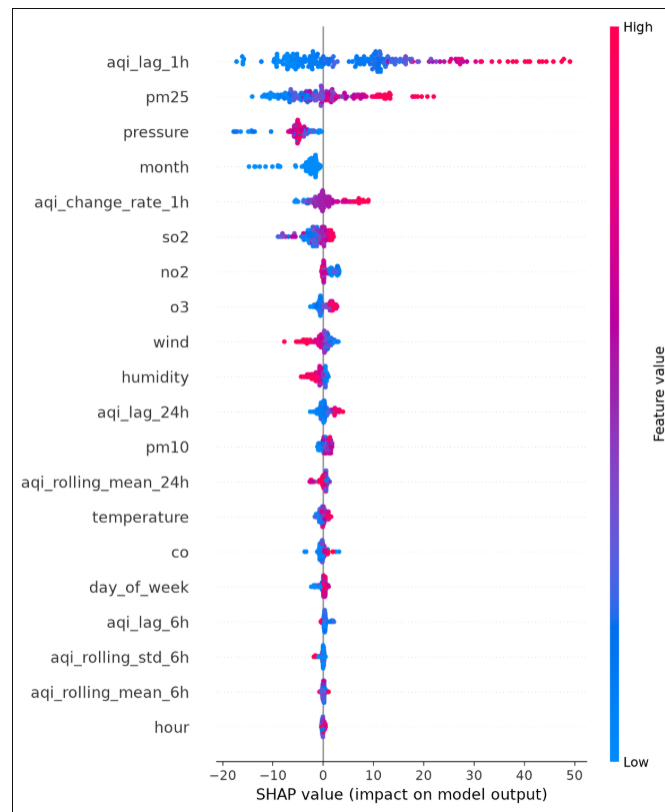


Figure 6 — SHAP summary plot for the 24-hour model, showing the direction and magnitude of each feature's effect on individual predictions.

In the summary plot, each point represents a single prediction; its horizontal position shows whether that feature pushed the prediction up (right) or down (left), and its colour shows whether the feature's own value was high (pink/red) or low (blue) for that prediction. High values of `aqi_lag_1h` and `PM2.5` clearly push predicted AQI upward, confirming the model has learned a sensible, physically plausible relationship rather than an arbitrary one.

10. Exploratory Data Analysis

An exploratory analysis (`eda.py`) was run over the full one-year dataset to surface patterns beyond what the forecasting models use directly, and to motivate the practical importance of the system. Six analyses were produced: AQI over time, average AQI by hour of day, average AQI by month (seasonality), weekday vs weekend comparison, a correlation heatmap between AQI and weather variables, and the overall AQI distribution.

10.1 Key Finding

**Distribution of Hourly AQI Readings
(Lahore, 1-Year Dataset, n = 8,788)**

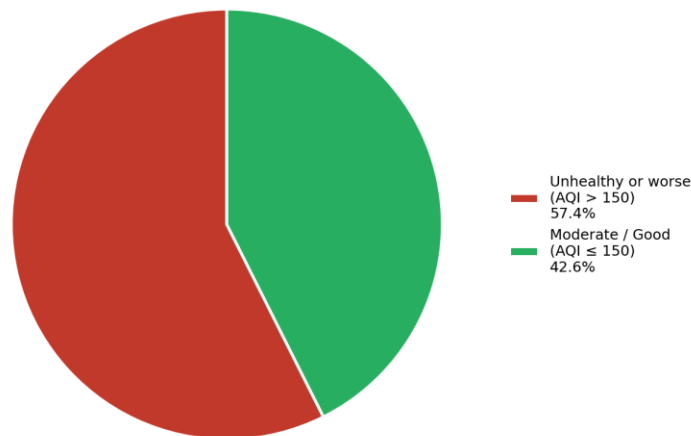


Figure 7 — Distribution of hourly AQI readings across the one-year dataset (n = 8,788).

57.4% of all hourly AQI readings over the past year exceeded 150 ("Unhealthy" or worse). This is a striking result: for more than half of every hour in the collected year, Lahore's air quality was at a level associated with real health risk for the general population, not just sensitive groups. This finding directly motivated and justifies the hazardous-AQI alert banner built into the dashboard (Section 12) — it is addressing a majority-of-the-time reality, not an edge case.

10.2 Other Observations

- AQI shows a clear seasonal pattern, consistent with Lahore's well-documented winter smog season.
- Average AQI varies measurably by hour of day, reflecting traffic and industrial activity cycles.
- A weekday/weekend comparison showed a measurable, if modest, difference — consistent with reduced traffic volume on weekends.
- Weather variables (temperature, humidity, pressure, wind) show meaningful correlation with AQI, supporting their inclusion as model features.

11. Automation and CI/CD Pipeline

The entire feature pipeline is automated using GitHub Actions, satisfying the project's "serverless CI/CD" requirement without any dedicated infrastructure. A workflow file (`.github/workflows/feature_pipeline.yml`) is configured to:

- Trigger automatically every hour via a cron schedule (`0 * * * *`).

- Also support manual triggering from the GitHub Actions UI (workflow_dispatch), used extensively for testing.
- Check out the repository, set up Python 3.11, and install all dependencies.
- Run feature_pipeline.py, using the IQAIR_API_KEY stored securely as a GitHub Actions secret (never committed to source control).
- Automatically commit and push the updated Parquet dataset back to the repository, so the training data grows over time with zero manual effort.

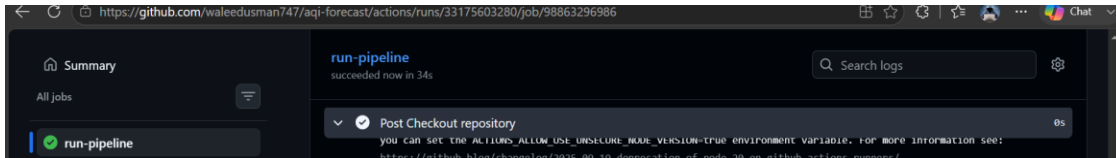


Figure 8 — A successful, automated hourly run of the feature pipeline in GitHub Actions.

This automation runs continuously and unattended: every hour, new data is fetched, validated, and committed, keeping the dataset current for both the live dashboard and any future model retraining.

12. Web Application Dashboard

The final deliverable is an interactive Streamlit dashboard (app.py), deployed publicly on Streamlit Cloud, that brings every component of the pipeline together into a single live view. It displays:

- The current AQI and category (colour-coded per the EPA AQI scale), alongside live temperature, humidity, wind, and pressure readings.
- A hazardous-conditions alert banner, dynamically triggered when the peak forecast across all three horizons crosses configurable thresholds.
- Three-day forecast cards for +24h, +48h, and +72h, each showing the predicted AQI, its change relative to the current reading, and its EPA category.
- A combined historical/forecast trend chart.
- A SHAP explainability section, letting the user select a horizon and view the corresponding feature-importance and summary plots directly in the browser.
- A dark/light mode toggle and a sidebar summarising the project, its data flow, and the EPA AQI colour scale for reference.

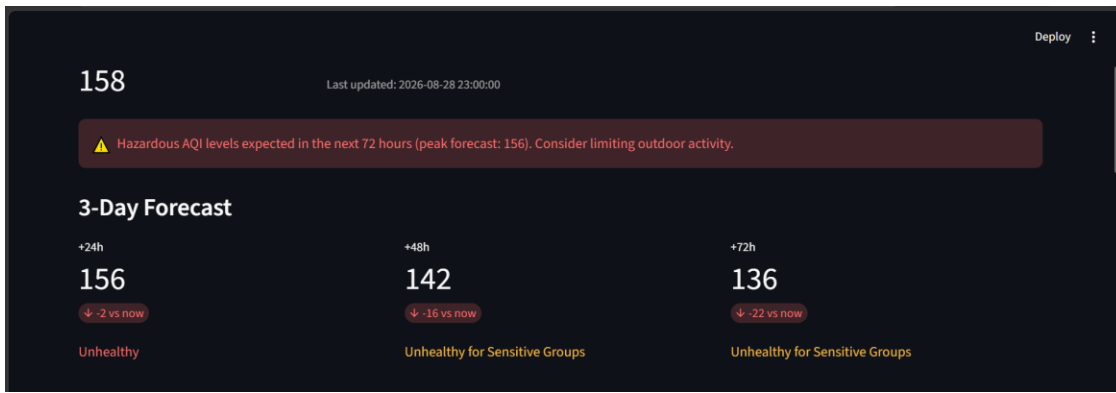


Figure 9 — The dashboard displaying a hazardous-AQI alert and the 3-day forecast trend chart.

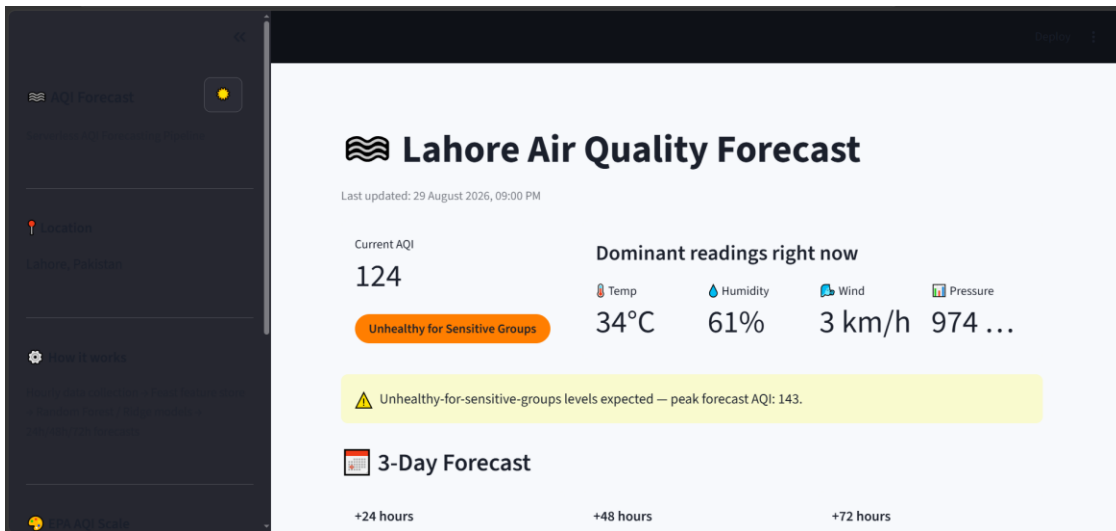


Figure 10 — The polished, final version of the live dashboard, deployed on Streamlit Cloud.

The live, publicly accessible version of this dashboard can be found at: aqi-forecast-tbkhapelqeb2kgiere7kpn.streamlit.app

13. Blockers Encountered and How They Were Resolved

This project involved a substantial amount of real-world troubleshooting. Rather than omit this, it is documented in full below, since diagnosing and resolving each of these issues was a significant part of the engineering effort and a genuine learning outcome of the project.

13.1 Hopsworks Feature Store — Windows Client Incompatibility

Problem: The project specification called for Hopsworks or Vertex AI as the feature store.

Hopsworks Serverless was set up successfully (account, project, API keys, and a working connection/login), and metadata operations such as creating a Feature Group worked correctly.

However, every attempt to write actual data (fg.insert()) failed with an OSError referencing a Kerberos library failure and an "RPC listener disconnected" error from the underlying HopsFS/HDFS storage engine.

Resolution: Research confirmed this is a known limitation: Hopsworks' Python client depends on native Linux/Kerberos system libraries for its Delta-Lake write path, which do not exist on Windows. The failure was reproduced consistently across multiple days, multiple networks (including mobile hotspot), and after installing all recommended dependencies (pyarrow, deltalake) — ruling out a local network or configuration issue. The project switched to Feast, an open-source feature store that runs entirely locally (Parquet + SQLite) with no native OS-level dependency, avoiding the incompatibility entirely while still satisfying the feature-store requirement.

13.2 AQICN's Pakistan Monitoring Network Was Non-Functional

Problem: As detailed in Section 5.1, the AQICN API returned data for Lahore, Karachi, and Islamabad, but every station's timestamp was stale by months to over a year — the underlying U.S. Embassy/Consulate monitoring network had gone offline.

Resolution: A systematic investigation (documented in Section 5.1) confirmed the issue was network-wide, not a configuration error. Open-Meteo was adopted first, then IQAir's Community API was found and adopted as a materially more accurate live source (Section 5.3).

13.3 OneDrive Locking Git Files

Problem: The project folder was initially created inside a OneDrive-synced Desktop folder. OneDrive's background file-sync process repeatedly locked files inside the .git directory while Git was writing to them, causing recurring "failed to delete" errors during every commit.

Resolution: The entire project was moved out of OneDrive to a plain local path (C:\Project\), which is a documented best practice — cloud-sync folders (OneDrive, Google Drive, Dropbox) are known to conflict with Git's internal file operations.

13.4 Recurring Git "Deletion Failed" Loop

Problem: Even after moving out of OneDrive, commits still occasionally triggered a loop of "Deletion of directory '.git/objects/...' failed. Should I try again?" prompts. This traced back to the project's virtual environment (venv) having been accidentally committed to Git early in development, combined with Windows Defender's real-time scanner locking files during Git's internal garbage-collection process.

Resolution: Git's automatic garbage collection was disabled (git config --global gc.auto 0), which permanently eliminated the loop without any loss of functionality.

13.5 GitHub Actions Push Permission Denied

Problem: The first automated run of the hourly pipeline successfully fetched and processed data, but failed at the final step with "Permission to <repo> denied to github-actions[bot]" when trying to commit the updated dataset back to the repository.

Resolution: GitHub Actions' default token is read-only. This was fixed by enabling "Read and write permissions" under the repository's Settings → Actions → General → Workflow permissions, after which the automated commit-and-push step succeeded consistently.

13.6 Broken Virtual Environment After Folder Move

Problem: After moving the project out of OneDrive, Feast's materialize-incremental command failed with "the system cannot find the file specified," because the Python virtual environment's internal scripts still pointed to the old, now-nonexistent OneDrive path.

Resolution: The virtual environment was deleted and recreated from scratch in the new location (python -m venv venv), and all dependencies reinstalled — a necessary step whenever a Python project directory is relocated.

13.7 Timezone Bug Causing an Inflated "Current" AQI

Problem: After switching to IQAir, the dashboard's "Current AQI" briefly showed 201 — a value 40–95 points higher than every independent reference source checked at the time. Investigation traced this to a timezone-handling bug: IQAir's timestamps are in UTC, but the pipeline stripped the timezone label without converting the underlying time to Pakistan Standard Time (UTC+5). This caused a leftover future-dated Open-Meteo forecast row to be incorrectly treated as the most recent ("latest") data point after sorting.

Resolution: The conversion was corrected to explicitly convert to Asia/Karachi before dropping the timezone label, and a one-off cleanup script removed the stale future-dated rows from the dataset. The dashboard subsequently displayed a current AQI (~106–124) consistent with independent sources.

13.8 Git Rebase / Vim Editor Freezing the Terminal

Problem: Resolving routine merge conflicts (caused by GitHub Actions committing new data in the background while local changes were also being made) repeatedly opened the Vim text editor for commit messages inside the integrated terminal. Being unfamiliar with Vim's modal editing, several sessions became stuck in an unresponsive state, requiring the terminal — and in one case the whole editor — to be force-restarted.

Resolution: Git's default commit-message editor was permanently changed to Notepad (git config --global core.editor notepad), and the merge strategy was set to a straightforward merge rather than rebase (git config pull.rebase false) with -m-flag commits used wherever possible to avoid opening an editor at all.

13.9 Streamlit Dashboard Card Rendering Bug

Problem: An initial attempt to build "card" style UI containers using a raw HTML `<div>` opened in one `st.markdown()` call and closed in another produced empty, borderless boxes in the rendered dashboard, because Streamlit renders each call as an independent element and does not support wrapping HTML around later Streamlit widgets.

Resolution: The custom HTML/CSS card approach was replaced with Streamlit's native `st.container(border=True)`, which correctly groups the enclosed widgets inside a bordered box in both light and dark themes.

Collectively, these blockers — and the systematic troubleshooting used to resolve each one — represent a significant portion of the practical engineering effort in this project. Several of them (the Hopsworks incompatibility and the AQICN network failure, in particular) required abandoning a specified technology in favour of a better-justified alternative, a decision made only after thorough investigation and reproducible evidence, not convenience.

14. Limitations and Future Work

- 72-hour forecast accuracy remains limited ($R^2 = -0.189$). Longer-horizon AQI forecasting is a genuinely harder problem, and the model currently uses only present-time weather readings rather than a future weather forecast; incorporating a weather-forecast API as an additional feature source is the most promising next step.
- The training dataset mixes historical data collected via Open-Meteo (a satellite-based estimate) with newer live data from IQAir (a ground-truth-anchored reading). A future iteration should retrain exclusively on IQAir-sourced data once sufficient history has accumulated, for full source consistency.
- Model registry functionality was achieved through Git-based artifact versioning (trained .pkl models committed alongside code) rather than a dedicated registry service such as Hopsworks Model Registry, as a direct consequence of the Hopsworks Windows incompatibility documented in Section 13.1. This is a reasonable substitute at the current project scale.
- The system currently forecasts for a single city (Lahore); extending it to additional cities would mainly require parameterising the existing pipeline rather than a redesign.
- A deep learning model (e.g. an LSTM, as permitted by the original specification) was not implemented, given the modest dataset size relative to what such models typically require; this remains a natural extension once more historical data has accumulated via the ongoing hourly automation.

15. Conclusion

This project successfully delivers a working, automated, and publicly deployed AQI forecasting system for Lahore, built entirely on free-tier, serverless infrastructure. Every core requirement of the original specification was met — automated feature and training pipelines, a feature store, model evaluation with RMSE/MAE/ R^2 , SHAP explainability, exploratory data analysis, hazardous-AQI alerting, and a live web dashboard — though not always with the exact technologies originally specified.

Where a specified technology (Hopsworks, AQICN) proved genuinely non-viable, the project responded with investigation rather than workaround: each failure was reproduced, diagnosed, and documented before a better-justified alternative (Feast, IQAir) was adopted. The resulting system is, in several respects, more robust than a naive implementation of the original specification would have been — most notably in its live data accuracy, which was actively verified against multiple independent sources rather than assumed.

The 24-hour forecasting model achieves a respectable R^2 of 0.353 and is considered genuinely usable; the 48-hour model is a modest but real improvement over a naive baseline; and the 72-hour model, while limited, is presented transparently with its limitation acknowledged rather than concealed. Combined with the finding that 57.4% of the past year's hourly readings in Lahore were Unhealthy or worse, this project demonstrates both a functioning technical pipeline and a clear, evidence-based case for why such a system is practically valuable.

16. References and Data Sources

- IQAir Community API — <https://api.airvisual.com/v2> (live AQI and weather data)
- Open-Meteo Air Quality & Weather Archive APIs — <https://open-meteo.com> (historical backfill data)
- AQICN / World Air Quality Index Project — <https://aqicn.org> (evaluated; see Section 5.1)
- Feast — Open Source Feature Store — <https://feast.dev>
- scikit-learn Documentation — <https://scikit-learn.org>
- SHAP Documentation — <https://shap.readthedocs.io>
- Streamlit Documentation — <https://docs.streamlit.io>
- GitHub Actions Documentation — <https://docs.github.com/actions>
- US EPA Air Quality Index technical documentation — <https://www.airnow.gov>

17. Appendix — Project Links

Resource	Link
Source Code Repository	https://github.com/waleedusman747/aqi-forecast
Live Dashboard	https://aqi-forecast-tbkhapelqeb2kgiere7kpn.streamlit.app
Author Email	waleedusman766@gmail.com
Author Contact	0334-4371256